

The logo of Gazi University is a circular seal. It features the university's name in Turkish, "GAZİ ÜNİVERSİTESİ", around the top inner edge and the year "1926" at the bottom. In the center is a stylized signature of "Gazi".

GAZİ UNIVERSITY

FACULTY OF ENGINEERING

DEPARTMENT OF INDUSTRIAL ENGINEERING

Lecturer : Dr Ercan Ezin

IE104-COMPUTER PROGRAMMING I

WEEK 13: EXERCISE FOR FINAL PROJECT

Exercise

We will implement Atbash Cipher in this class

Atbash Cipher

Write a **single-file C# console program** that encrypts and decrypts text using the **Atbash Cipher**.

In Atbash Cipher:

- A becomes Z
- B becomes Y
- C becomes X
- ...
- Z becomes A

The same operation is used for both encryption and decryption.

Example:

HELLO WORLD!

SVOOL DLIOW!

Your program must:

1.Show a menu:

1. Encode
2. Decode

2.Ask the user to choose input source:

1. Text input
2. File input

3.For text input, read multiple lines until an empty line is entered.

4.For file input, read all lines from the file path entered by the user.

5.Convert all output to uppercase.

6.Keep spaces, digits, punctuation, and symbols unchanged.

7.Print each processed line.

8.Print statistics after processing:

1. Total lines processed
2. Total characters
3. Total letters
4. Total digits
5. Total spaces
6. Total non-alphanumeric characters
7. Total words
8. Unique words
9. Most common letter
10. Least common letter among used letters

Dec	Hx	Oct	Char	Dec	Hx	Oct	Html	Chr	Dec	Hx	Oct	Html	Chr	Dec	Hx	Oct	Html	Chr
0	0	000	NUL (null)	32	20	040	 	Space	64	40	100	@	@	96	60	140	`	`
1	1	001	SOH (start of heading)	33	21	041	!	!	65	41	101	A	A	97	61	141	a	a
2	2	002	STX (start of text)	34	22	042	"	"	66	42	102	B	B	98	62	142	b	b
3	3	003	ETX (end of text)	35	23	043	#	#	67	43	103	C	C	99	63	143	c	c
4	4	004	EOT (end of transmission)	36	24	044	$	\$	68	44	104	D	D	100	64	144	d	d
5	5	005	ENQ (enquiry)	37	25	045	%	%	69	45	105	E	E	101	65	145	e	e
6	6	006	ACK (acknowledge)	38	26	046	&	&	70	46	106	F	F	102	66	146	f	f
7	7	007	BEL (bell)	39	27	047	'	'	71	47	107	G	G	103	67	147	g	g
8	8	010	BS (backspace)	40	28	050	(	(	72	48	110	H	H	104	68	150	h	h
9	9	011	TAB (horizontal tab)	41	29	051	)	)	73	49	111	I	I	105	69	151	i	i
10	A	012	LF (NL line feed, new line)	42	2A	052	*	*	74	4A	112	J	J	106	6A	152	j	j
11	B	013	VT (vertical tab)	43	2B	053	+	+	75	4B	113	K	K	107	6B	153	k	k
12	C	014	FF (NP form feed, new page)	44	2C	054	,	,	76	4C	114	L	L	108	6C	154	l	l
13	D	015	CR (carriage return)	45	2D	055	-	-	77	4D	115	M	M	109	6D	155	m	m
14	E	016	SO (shift out)	46	2E	056	.	.	78	4E	116	N	N	110	6E	156	n	n
15	F	017	SI (shift in)	47	2F	057	/	/	79	4F	117	O	O	111	6F	157	o	o
16	10	020	DLE (data link escape)	48	30	060	0	0	80	50	120	P	P	112	70	160	p	p
17	11	021	DC1 (device control 1)	49	31	061	1	1	81	51	121	Q	Q	113	71	161	q	q
18	12	022	DC2 (device control 2)	50	32	062	2	2	82	52	122	R	R	114	72	162	r	r
19	13	023	DC3 (device control 3)	51	33	063	3	3	83	53	123	S	S	115	73	163	s	s
20	14	024	DC4 (device control 4)	52	34	064	4	4	84	54	124	T	T	116	74	164	t	t
21	15	025	NAK (negative acknowledge)	53	35	065	5	5	85	55	125	U	U	117	75	165	u	u
22	16	026	SYN (synchronous idle)	54	36	066	6	6	86	56	126	V	V	118	76	166	v	v
23	17	027	ETB (end of trans. block)	55	37	067	7	7	87	57	127	W	W	119	77	167	w	w
24	18	030	CAN (cancel)	56	38	070	8	8	88	58	130	X	X	120	78	170	x	x
25	19	031	EM (end of medium)	57	39	071	9	9	89	59	131	Y	Y	121	79	171	y	y
26	1A	032	SUB (substitute)	58	3A	072	:	:	90	5A	132	Z	Z	122	7A	172	z	z
27	1B	033	ESC (escape)	59	3B	073	;	:	91	5B	133	[	[	123	7B	173	{	{
28	1C	034	FS (file separator)	60	3C	074	<	<	92	5C	134	\	\	124	7C	174	|	
29	1D	035	GS (group separator)	61	3D	075	=	=	93	5D	135	]	]	125	7D	175	}	}
30	1E	036	RS (record separator)	62	3E	076	>	>	94	5E	136	^	^	126	7E	176	~	~
31	1F	037	US (unit separator)	63	3F	077	?	?	95	5F	137	_	_	127	7F	177		DEL

SOLUTION

```
using System;
using System.IO;

class Program
{
    static void Main()
    { ... }

    static string[] ReadTextInput()
    { ... }

    static string TransformAtbash(string text)
    { ... }

    static void PrintStatistics(string[] lines)
    { ... }
}
```

```

static void Main()
{ // Main Starts
    Console.WriteLine("ATBASH CIPHER TOOLKIT");
    Console.WriteLine("1) Encode");
    Console.WriteLine("2) Decode");
    Console.Write("Choose mode: ");
    string mode = Console.ReadLine();

    if (mode != "1" && mode != "2")
    {
        Console.WriteLine("Invalid mode.");
        return;
    }

    Console.WriteLine("1) Text Input");
    Console.WriteLine("2) File Input");
    Console.Write("Choose source: ");
    string source = Console.ReadLine();

    string[] lines;

    if (source == "1")
    {
        lines = ReadTextInput();
    }
    else if (source == "2")
    {
        Console.Write("Enter file path: ");
        string path = Console.ReadLine();

        if (!File.Exists(path))
        {
            Console.WriteLine("File not found.");
            return;
        }

        lines = File.ReadAllLines(path);
    }
}

```

Continues



```

else
{
    Console.WriteLine("Invalid source.");
    return;
}

if (lines.Length == 0)
{
    Console.WriteLine("No input was provided.");
    return;
}

Console.WriteLine();
Console.WriteLine("OUTPUT");

string[] processedLines = new string[lines.Length];

for (int i = 0; i < lines.Length; i++)
{
    processedLines[i] = TransformAtbash(lines[i]);
    Console.WriteLine(processedLines[i]);
}

Console.WriteLine();
PrintStatistics(lines);
} // Main Ends

```

```
static string[] ReadTextInput()
{
    string[] temp = new string[1000];
    int count = 0;

    Console.WriteLine("Enter text lines. Enter an empty line to finish.");

    while (true)
    {
        string line = Console.ReadLine();

        if (line == "")
        {
            break;
        }

        temp[count] = line;
        count++;

        if (count == temp.Length)
        {
            break;
        }
    }

    string[] result = new string[count];

    for (int i = 0; i < count; i++)
    {
        result[i] = temp[i];
    }

    return result;
}
```

```
static string TransformAtbash(string text)
{
    text = text.ToUpper();
    string result = "";

    for (int i = 0; i < text.Length; i++)
    {
        char ch = text[i];

        if (ch >= 'A' && ch <= 'Z')
        {
            char newChar = (char)('Z' - (ch - 'A'));
            result += newChar;
        }
        else
        {
            result += ch;
        }
    }

    return result;
}
```



```

static void PrintStatistics(string[] lines)
{
    int totalCharacters = 0;
    int totalLetters = 0;
    int totalDigits = 0;
    int totalSpaces = 0;
    int totalNonAlphanumeric = 0;
    int totalWords = 0;
    int uniqueWordCount = 0;

    int[] letterCounts = new int[26];
    string[] words = new string[5000];

    for (int i = 0; i < lines.Length; i++)
    {
        string line = lines[i].ToUpper();
        totalCharacters += line.Length;

        string currentWord = "";

        for (int j = 0; j < line.Length; j++)
        {
            char ch = line[j];

            if (ch >= 'A' && ch <= 'Z')
            {
                totalLetters++;
                letterCounts[ch - 'A']++;
                currentWord += ch;
            }
            else
            {
                if (ch >= '0' && ch <= '9')
                {
                    totalDigits++;
                }

                if (ch == ' ')
                {
                    totalSpaces++;
                }
            }
        }
    }
}

```

```

        if (!char.IsLetterOrDigit(ch))
        {
            totalNonAlphanumeric++;
        }

        if (currentWord != "")
        {
            words[totalWords] = currentWord;
            totalWords++;
            currentWord = "";
        }
    }

    if (currentWord != "")
    {
        words[totalWords] = currentWord;
        totalWords++;
    }
}

for (int i = 0; i < totalWords; i++)
{
    bool foundBefore = false;

    for (int j = 0; j < i; j++)
    {
        if (words[i] == words[j])
        {
            foundBefore = true;
            break;
        }
    }

    if (!foundBefore)
    {
        uniqueWordCount++;
    }
}

```

```

char mostCommonLetter = '-';
char leastCommonLetter = '-';
int mostCount = 0;
int leastCount = int.MaxValue;

for (int i = 0; i < 26; i++)
{
    if (letterCounts[i] > mostCount)
    {
        mostCount = letterCounts[i];
        mostCommonLetter = (char)('A' + i);
    }

    if (letterCounts[i] > 0 && letterCounts[i] < leastCount)
    {
        leastCount = letterCounts[i];
        leastCommonLetter = (char)('A' + i);
    }
}

Console.WriteLine("STATISTICS");
Console.WriteLine("Total lines processed: " + lines.Length);
Console.WriteLine("Total characters: " + totalCharacters);
Console.WriteLine("Total letters: " + totalLetters);
Console.WriteLine("Total digits: " + totalDigits);
Console.WriteLine("Total spaces: " + totalSpaces);
Console.WriteLine("Total non-alphanumeric characters: " + totalNonAlphanumeric);
Console.WriteLine("Total words: " + totalWords);
Console.WriteLine("Unique words: " + uniqueWordCount);
Console.WriteLine("Most common letter: " + mostCommonLetter);
Console.WriteLine("Least common letter: " + leastCommonLetter);
}

```

THE END



CHECK OUT COURSE WEBSITE: [HTTPS://GAZI-END-MUH.GITHUB.IO](https://GAZI-END-MUH.GITHUB.IO)

GOT ANY QUESTIONS LATER?

SEND ME AN EMAIL: DR.ERCAN.EZIN@GMAIL.COM